Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 2:

Track B: Meet the OS You Will Build (xv6)

Elaborated: gr. FAF-243
Poiata Calin

Verified:
Patricia Reitman

Chișinău – 2026

Table of Contents

Environment & Toolchain 1
Part 1. Build It and Boot It 1
1. Compiling and Launching xv6 1
2. Exiting QEMU 1
Part 2. Use xv6 as the Unix It Is 1
Commands Tested Inside xv6 1
Observe (Part 2 Answers) 2
Part 3. Read the Source 2
1. Repository Structure: The Kernel/User Split 2
2. Tracing System Calls in user/cat.c 2
Observe (Part 3 Answers) 3
Part 4. Write Your First xv6 Program (sleep) 4
1. Implementation: user/sleep.c 4
2. Adding sleep to Makefile 4
3. Verification in xv6 4
Conclusions 5

Environment & Toolchain

- **Host OS:** Linux (Arch Linux x86_64, Kernel 7.2.7-arch1-1)
- **Target Architecture:** RISC-V 64-bit (`riscv64-linux-gnu`)
- **Emulator:** QEMU RISC-V System Emulator (`qemu-system-riscv64`)
- **Toolchain Packages:** `git` , `base-devel` , `gdb` , `qemu-system-riscv` , `riscv64-linux-gnu-gcc` , `riscv64-linux-gnu-binutils`

Part 1. Build It and Boot It

1. Compiling and Launching xv6

To compile the kernel, bootstrap loader, and user-space utilities, and then launch them under QEMU:

```
❱ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-
device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$
```

2. Exiting QEMU

xv6 executes inside the QEMU hardware emulator. To quit and return to the host terminal:

- Press `Ctrl-A` , release, then press `X` .

Part 2. Use xv6 as the Unix It Is

Commands Tested Inside xv6

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 32800
echo           2 4 31696
forktest       2 5 15856
grep           2 6 36248
init           2 7 32176
kill           2 8 31568
ln             2 9 31448
ls             2 10 35056
mkdir          2 11 31664
rm             2 12 31648
sh             2 13 54088
stressfs       2 14 32544
usertests      2 15 178872
grind          2 16 47688
wc             2 17 33496
zombie         2 18 31032
logstress      2 19 33544
forphan        2 20 31736
dorphan        2 21 31600
sync           2 22 31104
console        3 23 0

$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
```

```
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
...

$ echo hello xv6
hello xv6

$ ls | grep c
cat            2 3 32800
wc             2 17 33496
sync           2 22 31104
console        3 23 0

$ wc README
47 314 2441 README
```

Observe (Part 2 Answers)

1. **List three programs xv6 ships with:**
   - Three programs that ship with xv6 are `cat`, `echo`, and `grep`, among others (such as `sh`, `ls`, `rm`, `mkdir`, `wc`, and `kill`).
     - `ls`: Lists directory entries with file type, inode number, and size.
     - `cat`: Concatenates and displays file contents on standard output.
     - `echo`: Prints its command-line arguments to standard output.

2. **You just used a pipe (|) inside xv6. Which two OS features must exist for a pipe to work?**
   - For a pipe to work, two foundational operating system capabilities must exist:
     - **Process Creation and Scheduling (`fork()`):** We need processes (one on each side) running concurrently—a producer process (`ls`) and a consumer process (`grep`) that the OS schedules and context-switches.
     - **Inter-Process Communication (IPC via `pipe()`):** The OS must provide inter-process communication (the pipe itself)—an in-memory synchronized ring buffer managed by the kernel that binds standard output (file descriptor 1) of the writer to standard input (file descriptor 0) of the reader, handling flow control and blocking reads/writes.

3. **In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?**
   - xv6 is a way tiny version of the Linux shell that supports basic commands, pipes, and I/O redirection, but lacks modern conveniences like autocompletion (tab-completion), command history, environment variables, and job control.

Part 3. Read the Source

1. Repository Structure: The Kernel/User Split

The xv6 codebase cleanly separates privileged and unprivileged execution:

- `kernel/`: The operating system itself. Runs in privileged RISC-V Supervisor mode and has full control of the hardware, managing memory virtualization (page tables), process management, context switching, interrupt/exception handling, system calls, and the file system for all programs.
- `user/`: User-space applications (`cat`, `ls`, `echo`, `sh`, etc.). These are ordinary unprivileged programs that run in User mode, isolated from direct hardware access, requiring system calls to the kernel to perform privileged operations.

2. Tracing System Calls in user/cat.c

Inspecting `user/cat.c`:

```
void
cat(int fd)
{
  int n;
  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}
```

System calls are declared in `user/user.h` and dispatched to kernel routines:

- `user/user.h` declares function prototypes for user programs.
- `user/usys.pl` generates assembly stubs (`usys.S`) that load the system call number into register `a7` and execute `ecall` .
- `kernel/syscall.c` catches the trap, routes the call number, and calls the appropriate `sys_*` kernel handler.

Observe (Part 3 Answers)

1. **Which system calls does user/cat.c use, and what does each one ask the kernel to do?**
   - The calls used are `read` , `write` , `open` , `close` , and `exit` :
     - `open` : Asks the kernel to look up the file and give the process a new file descriptor.
     - `read` : Copy bytes from the open file `fd` into `buf` .
     - `write` : Copy bytes from `buf` into the open file / stream (`fd 1` for stdout).
     - `close` : Asks the kernel to release that file descriptor.
     - `exit` : Terminate the process and return the status.
2. **In technical kernel file, and on which line, is sys_read implemented?**
   - **File:** `kernel/sysfile.c`
   - **Line:** 69

```
uint64
sys_read(void)
{
  struct file *f;
  int n;
  uint64 p;

  argaddr(1, &p);
  argint(2, &n);
  if (argfd(0, 0, &f) < 0)
    return -1;
  return fileread(f, p, n);
}
```

3. **In one sentence: what is the difference between the code in kernel/ and the code in user/?**
   - Code in `kernel/` is in privileged mode and has full control of the hardware and manages stuff for everyone, whereas code in `user/` are ordinary unprivileged programs that require system calls to the kernel.

Part 4. Write Your First xv6 Program (sleep)

1. Implementation: user/sleep.c

The `sleep` utility was implemented in `user/sleep.c` to parse a command-line duration in clock ticks, validate user arguments, and invoke the kernel system call:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1])); // system call into the kernel
  exit(0);
}
```

2. Adding sleep to Makefile

In `Makefile`, `$U/_sleep\` was added to the `UPROGS` target list to ensure the utility is cross-compiled and included in the generated filesystem image (`fs.img`):

```
UPROGS=\
  $U/_cat\
  $U/_echo\
  $U/_sleep\
  $U/_forktest\
  $U/_grep\
  ...
```

3. Verification in xv6

The utility was compiled and verified inside the QEMU-emulated xv6 environment, testing both normal execution and argument validation:

```
$ sleep 10
(pauses for 10 timer ticks, then returns to prompt)
$ sleep
usage: sleep <ticks>
```

Conclusions

This laboratory work introduced the architecture, build environment, and execution model of the xv6 educational operating system running on the 64-bit RISC-V platform. By compiling and booting the kernel under QEMU, the fundamental mechanics of operating system initialization were demonstrated, from hardware bootstrap and multi-core hart activation to launching the first userspace init process and shell. Navigating the xv6 environment illustrated how classic Unix abstractions—including hierarchical file systems, standard stream redirection, and inter-process communication pipelines—are realized in a clean, minimal monolithic kernel without the overwhelming complexity of modern desktop operating systems.

Through systematic source code inspection of `kernel/` and `user/`, the critical boundary between privileged Supervisor mode and unprivileged User mode was examined in detail. Tracing utilities such as `cat` and inspecting system call implementations like `sys_read` in `kernel/sysfile.c` clarified the full lifecycle of a system call: user-space prototypes in `user/user.h`, assembly trap stubs in `usys.S` invoking `ecall`, kernel trap dispatching in `syscall.c`, and execution of privileged kernel handlers. This hands-on analysis reinforced the essential role of the kernel in enforcing hardware isolation, protecting physical memory via page tables, and mediating all interaction with peripheral devices.

Finally, developing and integrating the `sleep` utility provided valuable practical experience with low-level systems programming and cross-compilation toolchains. Implementing command-line argument validation and invoking the kernel's timer-based sleep system call demonstrated the smooth transition between user-space logic and privileged kernel services. Successfully integrating the new binary into the xv6 `Makefile` and verifying its execution under QEMU established a solid workflow for developing custom kernel extensions and user-space utilities in subsequent operating systems laboratories.